BOTO3 SERIES | LAB 3

EC2 Manager

Launch and control real virtual machines from Python — and destroy every one of them.

DURATION	SERVICES	DELIVERABLE	PATTERN
2 h	EC2, SSM	ec2_manager.py	Waiters

1 Goal

Build a menu-driven tool that lists, launches, starts, stops, reboots, describes, tags and terminates EC2 instances. It must use waiters rather than `time.sleep`, resolve the AMI ID from SSM rather than hardcoding it, and end with a cleanup command that terminates everything the lab created.

PREVIEW OF THE OUTPUT

```
========================================
EC2 MANAGER
========================================
1. List Instances        5. Reboot Instance
2. Launch Instance       6. Describe Instance
3. Start Instance        7. Add Tags
4. Stop Instance         8. Terminate Instance
9. CLEAN UP all lab instances
0. Exit
========================================
```

2 Learning objectives

- Name the four things every EC2 launch requires.
- Resolve the current Amazon Linux AMI ID from SSM instead of hardcoding it.
- Read `describe_instances()` correctly, including the `Reservations` nesting.
- Filter instances server-side, including by tag.
- Use waiters instead of `time.sleep` for every state change.
- Tag resources at launch, and use those tags to clean up afterwards.

3 Before you start

[ ] Labs 1 and 2 run successfully.

[ ] You have re-copied your credentials for this session.

[ ] You understand that instances cost money while running.

[ ] You have read the Learner Lab constraints below.

Learner Lab constraints

- Instance types are limited to nano, micro, small, medium and large. Use `t3.micro`: anything bigger fails with `UnauthorizedOperation`.
- Maximum 9 running instances and 32 vCPU per region. Launch one at a time.
- A key pair called `vockey` already exists in `us-east-1`. Pass `KeyName="vockey"`.
- EBS volumes are capped at 100 GB. The default 8 GB root volume is fine.
- Everything stops when the 4-hour session ends, but terminate your instances yourself anyway. That is the habit being taught.

4 Step by step

Build incrementally

Write `list_instances` first and keep it running in a second terminal. Being able to see the state of the world at all times makes the rest of this lab far easier.

Step 1: List instances, with a paginator

Use last session's paginator on `describe_instances`. Remember the double loop: instances live inside reservations.

Skip terminated instances in your output: they linger in the API for about an hour after termination and will confuse your list.

```
def list_instances():
    paginator = ec2.get_paginator("describe_instances")
    rows = []
    for page in paginator.paginate():
        for reservation in page["Reservations"]:
            for inst in reservation["Instances"]:
                if inst["State"]["Name"] != "terminated":
                    rows.append(inst)

    for inst in rows:
        tags = {t["Key"]: t["Value"] for t in inst.get("Tags", [])}
        print(f"{inst['InstanceId']:<21} "
              f"{inst['InstanceType']:<12} "
              f"{inst['State']['Name']:<12} "
              f"{inst.get('PublicIpAddress', '-'):<16} "
              f"{tags.get('Name', '(no name)')}")
```

Checkpoint

Each instance is listed with its ID, type, state, public IP and Name tag. Terminated instances do not appear, and an untagged instance prints `(no name)` instead of crashing.

Step 2: Resolve the AMI from SSM

Never hardcode an AMI ID. They differ per region and are retired every time Amazon publishes a patched image, so a copied ID is a bug with a delayed fuse.

AWS publishes the current ID as a public SSM parameter. This is how CloudFormation and Terraform do it too.

```
def get_latest_amazon_linux_ami():
    parameter = ("/aws/service/ami-amazon-linux-latest/"
                 "al2023-ami-kernel-default-x86_64")
    response = ssm.get_parameter(Name=parameter)
    return response["Parameter"]["Value"]
```

Checkpoint

Calling the function prints an ID starting with `ami-`. No AMI ID appears anywhere in your source code.

Step 3: Launch an instance, and tag it at launch

`run_instances` is the API behind the Console's launch wizard. Everything the wizard asks is a parameter here.

Use `TagSpecifications` rather than tagging afterwards: it is atomic, so the instance can never briefly exist untagged. That guarantee is what makes the cleanup in Step 7 reliable.

```
response = ec2.run_instances(
    ImageId=ami_id,
    InstanceType="t3.micro",
    MinCount=1,
    MaxCount=1,
    KeyName="vockey",
    TagSpecifications=[{
        "ResourceType": "instance",
        "Tags": [
            {"Key": "Name", "Value": name},
            {"Key": "CreatedBy", "Value": "boto3-lab"},
        ],
    }],
)
instance_id = response["Instances"][0]["InstanceId"]
print(instance_id, response["Instances"][0]["State"]["Name"])
```

> **Checkpoint**
>
> The Console shows a new instance in state `pending` or `running`, carrying both your `Name` tag and `CreatedBy = boto3-lab`.

Step 4: Wait properly

This is the pattern of the day. `run_instances` returned instantly, but the machine is not running yet. A waiter polls for you and raises `WaiterError` on timeout instead of failing silently.

`Delay` is the number of seconds between polls; `MaxAttempts` is how many times to poll. 15 x 40 gives a ten-minute ceiling.

```
from botocore.exceptions import WaiterError

try:
    print("Waiting for the instance to reach 'running' ...")
    waiter = ec2.get_waiter("instance_running")
    waiter.wait(
        InstanceIds=[instance_id],
        WaiterConfig={"Delay": 15, "MaxAttempts": 40},
    )
    print("Instance is running.")
except WaiterError:
    print("Timed out. Check the Console.")

# the public IP only exists once it is running
detail = ec2.describe_instances(InstanceIds=[instance_id])
running = detail["Reservations"][0]["Instances"][0]
print("Public IP:", running.get("PublicIpAddress", "not assigned"))
```

> **Checkpoint**
>
> The script blocks until the instance is running, then prints its public IP. Your code contains no `time.sleep`.

Step 5: Start, stop and reboot

Each call takes a list, so one call can operate on many instances. The response key differs per operation: `StartingInstances`, `StoppingInstances`, `TerminatingInstances`.

Print the previous and current state so the user can see the transition. Wait after start and stop; reboot has no waiter because the instance never leaves the `running` state.

```
r = ec2.start_instances(InstanceIds=[instance_id])
change = r["StartingInstances"][0]
print(change["PreviousState"]["Name"], "->",
      change["CurrentState"]["Name"])
ec2.get_waiter("instance_running").wait(InstanceIds=[instance_id])
```

> **Checkpoint**
>
> Stopping then starting an instance prints the previous and current state each time. Starting an already running instance gives `IncorrectInstanceState`, which your menu handles without a traceback.

Step 6: Describe one instance in full, and add a tag

Print the ID, name, state, type, AMI, availability zone, both IP addresses, key pair, launch time, security groups and every tag.

Then implement `create_tags`. It is idempotent: setting an existing key overwrites its value.

```
ec2.create_tags(
    Resources=[instance_id],
    Tags=[{"Key": "Owner", "Value": "youssef"}],
)
```

> **Checkpoint**
>
> Describing an instance shows every field listed above. After adding an `Owner` tag, describing it again shows the new tag.

Step 7: Clean up everything the lab created

This is the payoff for tagging at launch. Filter server-side on the `CreatedBy` tag and the instance state, collect the IDs, confirm, terminate, and wait.

Values within one filter are OR'd; separate filters are AND'd.

```
response = ec2.describe_instances(Filters=[
    {"Name": "tag:CreatedBy", "Values": ["boto3-lab"]},
    {"Name": "instance-state-name",
     "Values": ["pending", "running", "stopping", "stopped"]},
])
ids = [inst["InstanceId"]
       for res in response["Reservations"]
       for inst in res["Instances"]]

if ids and input("Type YES to confirm: ").strip() == "YES":
    ec2.terminate_instances(InstanceIds=ids)
    ec2.get_waiter("instance_terminated").wait(InstanceIds=ids)
    print(f"Terminated {len(ids)} instance(s).")
```

> **Checkpoint**
>
> Option 9 lists the instances it will terminate and asks for `YES`. Any other answer terminates nothing. After `YES`, the Console shows every lab instance as `terminated`.

5 Expected output

Your formatting may differ, but the information must be the same.

EXAMPLE SESSION

```
Choose an option: 2
Name tag for the instance: web-01
Using latest Amazon Linux 2023 AMI: ami-0abcd1234ef567890
Launched i-0a1b2c3d4e5f67890, state = pending
Waiting for the instance to reach 'running' ...
Instance is running.
Public IP: 54.87.123.45

Choose an option: 9
These instances will be terminated:
  i-0a1b2c3d4e5f67890
Type YES to confirm: YES
Waiting for all instances to terminate ...
Terminated 1 instance(s).
```

6 Troubleshooting

You see	It means	Do this
`UnauthorizedOperation`	Instance type not allowed here.	Use `t3.micro`. Larger types are blocked in the lab.
`InstanceLimitExceeded`	You have too many instances.	Terminate old ones. The lab allows 9.
`InvalidAMIID.NotFound`	The AMI does not exist in this region.	Resolve it from SSM, do not hardcode it.
`InvalidKeyPair.NotFound`	No key pair by that name.	Use `KeyName="vockey"` in `us-east-1`.
`WaiterError`	The state was not reached before the timeout.	Raise `MaxAttempts`, or check the Console.
`InvalidInstanceID.NotFound`	Wrong ID, or wrong region.	Copy the ID from your own list output.
`IncorrectInstanceState`	Wrong action for the current state.	You cannot start a running instance.

7 Cleanup

This is the lab where cleanup genuinely matters. An instance left running overnight spends lab credit that belongs to everyone in the room.

- [ ] Option 9 has been run and reported success.
- [ ] The EC2 Console shows every instance in the `terminated` state.
- [ ] No volumes remain under Elastic Block Store > Volumes.
- [ ] Your script is committed to your Git repository.

8 Submission checklist

- [ ] `ec2_manager.py` runs end to end with no traceback.
- [ ] All nine menu options work: list, launch, start, stop, reboot, describe, add tags, terminate, clean up.
- [ ] The AMI ID comes from SSM and every state change uses a waiter, with no `time.sleep`.
- [ ] Instances are tagged at launch with `CreatedBy = boto3-lab`, and option 9 terminates them all.
- [ ] The file is committed with a meaningful message.